Tugas 2 : Kecerdasan Buatan

Soal Essay: Penyelesaian Masalah dengan Teknik Pencarian

Soal 1: Formulasi Masalah (Problem Formulation) Dalam Kecerdasan Buatan, sebelum sebuah algoritma pencarian dijalankan, masalah harus dirumuskan terlebih dahulu ke dalam struktur yang dapat dipahami mesin. Sebutkan dan jelaskan 6 komponen utama dalam perumusan masalah (problem formulation)! Berikan contoh penerapannya pada kasus "Robot pengantar barang dari titik A ke titik B di dalam sebuah gudang".

Soal 2: Pencarian Buta (Uninformed Search) Pencarian buta (uninformed search atau blind search) tidak menggunakan informasi tambahan tentang perkiraan jarak menuju tujuan. Bandingkan dua algoritma pencarian buta, yaitu Breadth-First Search (BFS) dan Depth-First Search (DFS)! a. Jelaskan perbedaan cara kerja kedua algoritma tersebut! b. Jelaskan kelebihan dan kekurangan masing-masing algoritma dari segi kompleksitas waktu, kompleksitas ruang (memori), dan kelengkapan (completeness)!

Soal 3: Iterative Deepening Search (IDS) Algoritma Iterative Deepening Search (IDS) sering disebut sebagai algoritma hybrid yang menggabungkan kelebihan dari BFS dan DFS. Jelaskan bagaimana cara kerja IDS, dan mengapa IDS sering menjadi pilihan utama untuk pencarian ruang status yang sangat luas (seperti pohon catur) ketika konsumsi memori menjadi pertimbangan utama!

Soal 4: Pencarian Heuristik dan Algoritma A* Pencarian dengan informasi heuristik (informed search) menggunakan fungsi heuristik, h(n), untuk memperkirakan biaya menuju tujuan. a. Jelaskan apa yang dimaksud dengan fungsi heuristik yang Admissible (dapat diterima)! Mengapa sifat admissible sangat penting dalam algoritma A* (A-Star)? b. Algoritma A* menggunakan persamaan f(n) = g(n) + h(n). Jelaskan makna dari f(n), g(n), dan h(n) tersebut!

Soal 5: Pencarian Lokal (Local Search) dan Optimasi Algoritma pencarian lokal seperti Hill Climbing sering digunakan untuk masalah optimasi yang ruang statusnya terlalu besar untuk disimpan dalam memori (misal: Travelling Salesperson Problem atau TSP). a. Jelaskan cara kerja dasar algoritma Hill Climbing! b. Apa kelemahan utama dari Hill Climbing standar? Sebutkan dan jelaskan satu variasi algoritma yang dapat mengatasi kelemahan tersebut (misalnya: Simulated Annealing atau Genetic Algorithm)!

Soal 6: Studi Kasus - N-Queens Problem Masalah N-Queens menuntut penempatan N buah ratu pada papan catur berukuran NxN sedemikian rupa sehingga tidak ada dua ratu yang saling mematikan (satu baris, kolom, atau diagonal). Jika Anda ditugaskan menyelesaikan masalah 8-Queens menggunakan teknik pencarian, mana yang lebih efisien digunakan: algoritma pencarian pohon ruang status (tree search seperti BFS/DFS) atau algoritma pencarian lokal (seperti Min-Conflicts)? Berikan argumentasi ilmiah mengenai pilihan Anda!

Soal 7: Analisis Dilema "Time vs Space Trade-off" Sebuah perusahaan ride-hailing ingin membuat sistem AI yang dapat mencari rute terpendek bagi pengemudi ojek online untuk mengantarkan penumpang. Sistem ini harus memproses jutaan rute secara real-time. Berdasarkan karakteristik algoritma pencarian (BFS, DFS, A*, IDS, Hill Climbing), algoritma mana yang paling tidak relevan atau akan menyebabkan kegagalan sistem jika diterapkan mentah-mentah pada skala besar ini? Berikan alasan mengapa algoritma tersebut tidak cocok, dan sebutkan algoritma alternatif yang seharusnya digunakan beserta alasan teknisnya!

Petunjuk Pengerjaan :

1. Kerjakan secara Kelompok
2. Buat pada Blog atau Web pribadi bukan Google Drive.
3. Setiap anggota kelompok WAJIB mengirimkan jawaban dengan menggunakan LINK dari Blog yang dipilih.
4. Cantumkan NIM, Nama Lengkap pada jawaban yang diberikan
5. Kerjakan dengan Analisa yang baik dan terstruktur, serta dilengkapi dengan sumber referensi yang digunakan (Boleh menggunakan bantuan AI, ChatGPT dan sejenisnya, namun harus diperkuat dengan hasil pemikiran sendiri).
6. Jawaban dapat disertai dengan gambar atau diagram pendukungnya.
7. Muat hasil pengerjaan pada Link di MyBest

JAWABAN

TUGAS 2: KECERDASAN BUATAN Topik: Penyelesaian Masalah dengan Teknik Pencarian

Identitas Kelompok:

1. Razaan Kurniawan Putra (NIM: 17240286)
2. Ridho Putra Rianar (NIM: 17240102)
3. Aby Yazzy Wijaya (NIM: 17240158)
4. Toma Brema (NIM: 17240099)
5. Sam Nurullah (NIM: 17240117)

==========

SOAL 1: FORMULASI MASALAH (PROBLEM FORMULATION)

Perumusan masalah formal dalam lingkungan agen cerdas terdiri dari 6 komponen utama. Berikut adalah penjelasan dan penerapannya pada kasus "Robot pengantar barang dari titik A ke titik B di dalam gudang":

1. Initial State (Kondisi Awal) Keadaan atau konfigurasi lingkungan saat agen pertama kali memulai proses pencarian. Contoh Kasus: Robot berada diam di titik koordinat awal A (misalnya dermaga bongkar muat / loading dock) membawa muatan barang yang siap dikirim.
2. Actions (Himpunan Tindakan) Daftar semua kemungkinan aksi legal atau perpindahan yang diizinkan untuk dieksekusi oleh agen dari suatu keadaan. Contoh Kasus: Robot dapat bergerak 1 meter Maju, Mundur, Belok Kiri 90 derajat, Belok Kanan 90 derajat, atau Berhenti.
3. Transition Model (Model Transisi / Result Function) Deskripsi formal mengenai hasil luaran dari eksekusi suatu aksi pada status tertentu: Result(s, a) = s'. Contoh Kasus: Jika robot berada pada koordinat (X, Y) menghadap ke utara dan mengeksekusi aksi "Maju 1 meter",

maka status berikutnya berpindah ke koordinat (X, Y+1) menghadap ke utara, asalkan tidak menabrak rak gudang atau rintangan.

4. Goal Test (Uji Kondisi Tujuan) Fungsi atau kondisi penentu apakah status saat ini sudah memenuhi kriteria target akhir yang diinginkan. Contoh Kasus: Sistem memeriksa apakah koordinat posisi robot saat ini identik dengan koordinat titik B (misalnya rak penyimpanan zona B-12). Jika benar, misi selesai.
5. Path Cost (Biaya Jalur / Biaya Langkah) Fungsi biaya numerik yang mencerminkan beban dari urutan aksi yang diambil untuk mencapai tujuan, dinotasikan c(s, a, s'). Contoh Kasus: Total waktu tempuh (dalam detik), konsumsi daya baterai robot (dalam Watt-hour), atau total jarak tempuh fisik (dalam meter) sepanjang rute dari A ke B.
6. State Space (Ruang Status) Kumpulan seluruh kemungkinan status atau konfigurasi posisi yang dapat dicapai robot dari initial state melalui kombinasi aksi yang sah. Contoh Kasus: Seluruh peta grid lantai gudang yang dapat dilalui (area bebas halangan rak, dinding pembatas, atau jalur konveyor).

==========

SOAL 2: PENCARIAN BUTA (UNINFORMED SEARCH)

a. Perbedaan Cara Kerja BFS dan DFS

- Breadth-First Search (BFS): Menjelajahi ruang status lapis demi lapis secara horizontal (level by level). Algoritma ini mengekspansi semua simpul pada kedalaman d sebelum melangkah ke simpul-simpul pada kedalaman d+1. BFS diimplementasikan menggunakan struktur data antrean FIFO (First-In, First-Out).
- Depth-First Search (DFS): Menjelajahi ruang status sedalam mungkin ke satu cabang cabang secara vertikal sebelum melakukan penelusuran balik (backtracking). Algoritma ini mengekspansi simpul anak paling dalam terlebih dahulu hingga menemui jalan buntu (dead-end). DFS diimplementasikan menggunakan struktur data tumpukan LIFO (Last-In, First-Out) atau fungsi rekursif.

b. Perbandingan Karakteristik (b = branching factor, d = kedalaman solusi dangkal, m = kedalaman maksimum pohon)

1. Kompleksitas Waktu (Time Complexity):

- BFS: O(b^d). Waktu bertambah secara eksponensial terhadap kedalaman solusi.
- DFS: O(b^m). Jika pohon sangat dalam atau memiliki cabang tak hingga, waktu pencarian dapat jauh lebih lama dibanding BFS jika solusi berada di cabang lain.

2. Kompleksitas Ruang / Memori (Space Complexity):

- BFS: O(b^d). Sangat boros memori karena seluruh simpul yang dibangkitkan pada level saat ini dan level sebelumnya harus tetap disimpan di RAM dalam queue.
- DFS: O(b * m). Sangat hemat memori karena hanya perlu menyimpan jalur aktif dari akar (root) menuju daun (leaf) beserta saudara-saudaranya yang belum diekspansi.

3. Kelengkapan (Completeness):

- BFS: Complete. Dijamin pasti menemukan solusi jika solusi memang ada (asalkan branching factor b bernilai hingga).
- DFS: Incomplete pada ruang status tak terhingga atau grafik yang memiliki siklus/loop (bisa terjebak dalam infinite loop jika tidak dilengkapi pengecekan visited nodes). Lengkap hanya jika ruang status berhingga dan memiliki deteksi siklus.

==========

SOAL 3: ITERATIVE DEEPENING SEARCH (IDS)

Cara Kerja Iterative Deepening Search (IDS): IDS bekerja dengan mengeksekusi algoritma Depth-Limited Search (DFS dengan batas kedalaman) secara berulang-ulang, dimulai dari batas kedalaman (depth limit) L = 0, L = 1, L = 2, dan seterusnya secara bertahap dinaikkan sampai solusi pertama kali ditemukan. Pada setiap iterasi baru, pencarian diulang kembali dari simpul akar hingga mencapai batas kedalaman L yang baru.

Alasan IDS Menjadi Pilihan Utama untuk Ruang Status Sangat Luas (seperti Pohon Catur):

1. Efisiensi Memori Selevel DFS: IDS mempertahankan kompleksitas memori linear yaitu O(b * d), di mana b adalah faktor percabangan dan d adalah kedalaman solusi. Pada permainan seperti catur dengan branching factor sekitar b ≈ 35, BFS akan langsung menghabiskan seluruh kapasitas RAM komputer hanya pada kedalaman ke-5 atau ke-6. IDS dapat berjalan lancar tanpa mengalami masalah kehabisan memori (Out-of-Memory).
2. Jaminan Kelengkapan dan Optimalitas Selevel BFS: Karena kedalaman bertambah bertahap (0, 1, 2, ...), IDS dijamin complete dan optimal (menemukan solusi dengan jumlah langkah paling sedikit jika biaya tiap langkah sama), mirip dengan karakteristik BFS.
3. Biaya Overhead Komputasi Ulang yang Relatif Rendah: Meskipun simpul level atas di-generate berulang kali, sebagian besar simpul dalam pohon percabangan berada pada daun level terbawah. Proporsi komputasi ulang simpul level atas hanya menambah overhead kecil (sekitar b/(b-1)), yang sangat terbayar dengan penghematan memori yang masif.

==========

SOAL 4: PENCARIAN HEURISTIK DAN ALGORITMA A*

a. Heuristik yang Admissible dan Urgensinya dalam Algoritma A*

- Definisi Admissible: Suatu fungsi heuristik h(n) dikatakan admissible (dapat diterima) apabila nilai perkiraan biaya dari simpul n menuju tujuan tidak pernah melebihi biaya sebenarnya (never overestimates the true cost). Secara matematis dituliskan: h(n) <= h*(n), di mana h*(n) adalah biaya riil terpendek menuju tujuan. Nilai h(Goal) harus sama dengan 0.
- Urgensi Sifat Admissible: Sifat ini menjamin bahwa algoritma A* bersifat Optimal (pasti menemukan jalur berbiaya terendah). Jika heuristik membesar-besarkan biaya (overestimate), A* dapat mengabaikan jalur terbaik yang sebenarnya murah hanya karena heuristiknya keliru memprediksi bahwa jalur tersebut mahal, sehingga algoritma berakhir pada solusi suboptimal.

b. Makna Komponen f(n) = g(n) + h(n)

- g(n): Biaya Historis (Past/Exact Cost). Merupakan biaya riil yang telah dihabiskan untuk melakukan perjalanan dari simpul awal (start node) hingga ke simpul saat ini n.

- h(n): Biaya Perkiraan Masa Depan (Future/Estimated Cost). Merupakan fungsi perkiraan (heuristik) dari simpul saat ini n menuju simpul tujuan (goal node).
- f(n): Estimasi Biaya Total Jalur (Total Estimated Path Cost). Merupakan taksiran total biaya keseluruhan perjalanan termurah yang melewati simpul n dari awal hingga ke tujuan. Algoritma A* selalu memprioritaskan ekspansi simpul yang memiliki nilai f(n) terkecil pada antrean prioritas (priority queue).

==========

SOAL 5: PENCARIAN LOKAL (LOCAL SEARCH) DAN OPTIMASI

a. Cara Kerja Dasar Algoritma Hill Climbing Hill Climbing (sering disebut steepest-ascent) bekerja mirip dengan pendaki gunung yang mendaki dalam kabut tebal:

1. Algoritma memulai dari satu kondisi awal acak (current state).
2. Mengevaluasi seluruh status tetangga (neighbor states) di sekitarnya menggunakan fungsi evaluasi objektif/fitness.
3. Memilih tetangga dengan nilai evaluasi tertinggi (perbaikan terbaik dibanding status saat ini).
4. Jika tetangga terbaik tersebut lebih baik daripada status saat ini, agen berpindah ke tetangga tersebut dan menjadikannya status baru.
5. Proses diulang terus-menerus hingga tidak ada lagi tetangga yang memiliki nilai lebih tinggi dari status saat ini. Pada titik ini, algoritma berhenti dan menganggap posisi tersebut sebagai solusi terbaik.

b. Kelemahan Utama dan Solusi Alternatif

- Kelemahan Utama: Terjebak pada Local Optima (Puncak Lokal). Algoritma ini berhenti saat berada di puncak bukit lokal karena semua tetangga di sekitarnya memiliki nilai lebih rendah, padahal ada puncak global (Global Optimum) yang jauh lebih tinggi di bagian lain ruang status. Masalah lain mencakup plateau (wilayah datar tanpa kemiringan) dan ridges (punggung bukit tajam).
- Solusi Variasi: Algoritma Simulated Annealing. Penjelasan: Terinspirasi dari proses pendinginan logam (annealing) dalam metalurgi. Algoritma ini tidak hanya mengambil langkah yang membaik, tetapi juga mengizinkan "langkah buruk" (downhill moves) dengan probabilitas tertentu untuk meloloskan diri dari jebakan puncak lokal. Probabilitas menerima langkah buruk diatur oleh parameter suhu T (Temperature). Di awal pencarian, nilai T tinggi sehingga algoritma bebas melompat menjelajahi area baru. Seiring berjalannya waktu, suhu T menurun secara bertahap menuju 0, membuat algoritma semakin selektif hingga berperilaku murni seperti Hill Climbing saat mendekati konvergensi di global optimum.

==========

SOAL 6: STUDI KASUS - N-QUEENS PROBLEM

Rekomendasi Pilihan: Algoritma Pencarian Lokal (seperti Min-Conflicts / Local Search) jauh lebih efisien dibandingkan Algoritma Pencarian Pohon (Tree Search seperti BFS/DFS).

Argumentasi Ilmiah:

1. Ukuran Ruang Status dan Ledakan Kombinatorial (Combinatorial Explosion): Pada papan 8-Queens, jika diselesaikan dengan Tree Search (misal DFS murni tanpa pruning lanjutan), ruang status penempatan 8 ratu pada 64 petak mencapai 64! / (8! * 56!) ≈ 4,4 x 10^9 kombinasi. Bahkan jika dibatasi satu ratu per kolom, ruang statusnya tetap 8^8 = 16.777.216 kemungkinan status. Melacak status pohon membutuhkan pembentukan struktur data tree/graph yang berat dan memerlukan waktu traversal yang lama.

2. Karakteristik Masalah (Goal State vs Path to Goal): N-Queens adalah masalah kepuasan batasan (Constraint Satisfaction Problem). Yang dicari hanyalah *konfigurasi akhir papan* di mana tidak ada ratu yang saling serang. *Jalur atau urutan langkah* bagaimana ratu ditempatkan sama sekali tidak penting. Algoritma Tree Search (BFS/DFS) membuang-buang memori dan daya komputasi hanya untuk menyimpan urutan rantai status (path history) yang tidak bernilai.

3. Efisiensi Komputasi Algoritma Min-Conflicts: Algoritma Min-Conflicts memulai dari penempatan lengkap 8 ratu (satu ratu per kolom secara acak). Pada setiap langkah iterasi:

- Algoritma memilih secara acak satu ratu yang mengalami konflik (saling serang).
- Memindahkan ratu tersebut pada kolom yang sama ke baris yang meminimalkan jumlah konflik serangan. Secara empiris, algoritma Min-Conflicts dapat menyelesaikan 8-Queens dalam rata-rata kurang dari 50 langkah (orde O(1) hingga O(N)). Faktanya, Min-Conflicts bahkan dapat menyelesaikan N-Queens untuk N = 1.000.000 (satu juta ratu) dalam hitungan detik, sesuatu yang mustahil dilakukan oleh BFS atau DFS.

==========

SOAL 7: ANALISIS DILEMA "TIME VS SPACE TRADE-OFF"

1. Algoritma yang Paling Tidak Relevan / Menyebabkan Kegagalan Sistem: Algoritma Breadth-First Search (BFS) dan Depth-First Search (DFS) murni. Secara khusus, BFS adalah yang paling fatal menyebabkan kegagalan sistem seketika (crash sistem).

Alasan Mengapa Tidak Cocok:

- Ledakan Memori Ekstrem pada BFS (Out of Memory Failure): Peta jalan raya kota pada aplikasi ride-hailing dimodelkan sebagai graf berbobot raksasa dengan jutaan persimpangan (nodes) dan ruas jalan (edges). BFS mengekspansi seluruh simpul bertingkat secara meluas. Dengan jutaan permintaan perjalanan real-time, antrean FIFO BFS akan menampung jutaan node jalan raya di RAM per request. Server akan mengalami Out-Of-Memory (OOM) dalam hitungan detik. Selain itu, BFS mengasumsikan biaya tiap ruas jalan adalah sama (bobot 1), padahal jarak jalan dan durasi macet di dunia nyata sangat variatif.

- DFS Menghasilkan Rute Irasional (Inoptimal): DFS akan menelusuri satu ruas jalan hingga ke ujung pulau/kota lain sebelum melakukan backtracking. Hal ini menghasilkan rute yang sangat berputar-putar, boros bahan bakar, dan sama sekali tidak menjamin jarak terpendek.

- Hill Climbing Menghasilkan Rute Buntu: Hill Climbing mudah terjebak di jalan buntu (dead-end) atau persimpangan satu arah karena hanya melihat perbaikan jarak satu langkah ke depan secara serakah tanpa memikirkan jaringan jalan secara global.

2. Algoritma Alternatif yang Seharusnya Digunakan: Algoritma A* Search yang dioptimasi (khususnya Bidirectional A* Search atau Contraction Hierarchies berbasis A*).

Alasan Teknis:

- Efisiensi Heuristik Terarah: A* menggunakan kombinasi f(n) = g(n) + h(n). Nilai g(n) mencatat jarak tempuh/waktu nyata dari titik penjemputan, sementara h(n) menggunakan jarak garis lurus (Euclidean / Haversine distance) menuju titik tujuan sebagai heuristik yang admissible. Hal ini membuat pencarian fokus mengarah ke tujuan dan tidak menyebar ke segala arah seperti BFS/Dijkstra standar, memangkas ruang pencarian secara drastis.
- Jaminan Optimalitas Rute: Karena heuristik jarak garis lurus tidak pernah melebihi jarak jalan riil (admissible), rute yang dihasilkan dijamin merupakan rute terpendek / tercepat secara matematis.
- Skalabilitas Industri (Contraction Hierarchies): Dalam sistem ride-hailing berskala produksi, algoritma A* dikombinasikan dengan teknik pra-pemrosesan seperti Contraction Hierarchies (CH). Jalan-jalan utama (arteri/tol) diberi prioritas hierarki lebih tinggi daripada gang perumahan, sehingga rute jutaan titik antarkota dapat dihitung hanya dalam waktu beberapa milidetik dengan penggunaan memori yang sangat minim.